
Recursive Language Models and Slumbot for Poker Decision-Making

Aryan Arora¹ Aarav Mulinti¹ Aadithya Srinivasan¹ Aryan Jeena¹

Abstract

We present a two-part system for poker decision-making: a Recursive Language Model (RLM) that generates executable Python against a sandboxed REPL, and a Slumbot integration that provides a near-Nash opponent for real-world evaluation. The RLM is fine-tuned in two stages — behavior cloning on 500 heuristic trajectories, then REINFORCE with batch-normalized clipped advantages — starting from Qwen2.5-Coder-1.5B. A zero-shot Qwen-7B baseline achieves 8% action-type accuracy; BC initialization reaches 25%; 130 RL iterations lift the EMA to a peak of **42.3%** before reward-hacking collapse. A shaped-reward variant that explicitly incentivizes REPL usage recovers to **33.3%** held-out accuracy with real code generation stabilized at 95% of rollouts, demonstrating that output-only rewards are insufficient for training code-generating agents.

1. Introduction

Optimal poker play requires exploiting opponent tendencies visible only across many prior hands — VPIP, PFR, fold-to-continuation-bet, and aggression factors that every hand-history tracker computes. Standard LLM prompting cannot handle the full context: each episode contains the current hand state plus 15 prior hand records, averaging 4,300 characters. The Recursive Language Model (RLM) framework (Zaremba et al., 2025) addresses this by letting the model *write code* that parses the context in a sandboxed Python REPL and prints the final action.

This project has two complementary components: **(1) RLM poker** — a Qwen2.5-Coder-1.5B model trained via behavior cloning then REINFORCE to reason about hands by generating and executing Python; and **(2) Slumbot integration** — the trained RLM playing live heads-up hands against Slumbot (Jackson, 2013), the 2018 ACPC champion, providing an opponent-independent real-world benchmark.

Our contributions are:

1. A reproducible poker-RLM stack (environment, heuristic

expert, BC+RL trainer, evaluation framework) in <3k lines of Python, runnable end-to-end in <2 hours on one H100.

2. An empirical isolation of what each training stage contributes (zero-shot $\rightarrow$ BC $\rightarrow$ RL), with hardware and seed fixed.
3. A diagnostic study of reward hacking in code-generating RL agents and a shaped-reward remedy that eliminates the bypass attractor.
4. A live Slumbot bridge demonstrating that the RLM pattern (LLM $\rightarrow$ code $\rightarrow$ REPL $\rightarrow$ action) plugs into a real poker-server protocol under genuine distribution shift.

2. Background

Recursive Language Models. Zaremba et al. (2025) showed that an LLM with Python REPL access can answer needle-in-haystack queries over long structured text by writing extraction code rather than attending over all tokens. We extend this from retrieval to sequential decision-making.

REINFORCE with baseline. We use the log-derivative policy gradient estimator with an EMA baseline and normalized, clipped advantages:

$$\hat{A}_i = \text{clip}\left(\frac{r_i - b}{\sigma_r}, -2, +2\right), \quad b \leftarrow 0.95b + 0.05\bar{r}. \quad (1)$$

The policy-gradient loss for rollout i is $\mathcal{L}_i = -\hat{A}_i \sum_t \log p_\theta(y_t | x, y_{<t})$. This batch-normalized clipping is more stable than raw REINFORCE at batch size 4 (Ahmadian et al., 2024).

LoRA fine-tuning. We apply low-rank adaptation (Hu et al., 2022) with $r=16$, $\alpha=32$ to the attention projection matrices only, keeping 99.5% of the base model frozen. This allows fine-tuning on a single GPU in 4-bit NF4 precision.

Retrospective checkpoint selection. Following standard practice, we save a checkpoint every 5 RL iterations and select the one with the highest held-out EMA reward for evaluation, rather than using the final checkpoint.

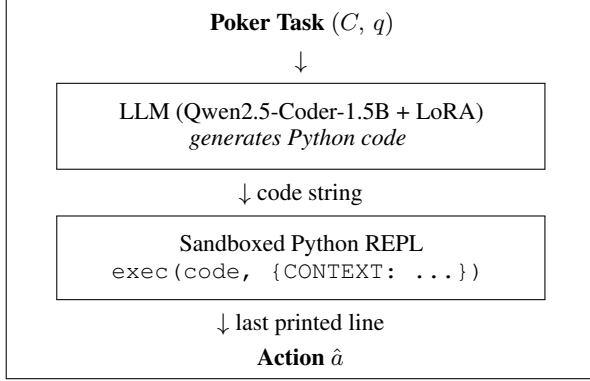


Figure 1. RLM pipeline. The long context lives in a Python environment. The model emits code; the sandbox executes it and returns the action.

3. System Design

3.1. Task Formulation

Each episode is a tuple (C, q, a^*) where C is the game-state context (hole cards, board, stacks, positions, current pot/betting, and 15 structured prior-hand records), q is a fixed question, and a^* is the heuristic expert’s action. The agent observes (C, q) , emits Python code, the sandbox executes it, and the last printed line is parsed as $\hat{a} \in \{\text{fold, check, call } \$X, \text{raise } \$X\}$. The primary reward is action-type match: $r=1$ if $\text{type}(\hat{a}) = \text{type}(a^*)$, else $r=0$.

3.2. Poker Environment

A 6-max No-Limit Hold’em environment provides card objects, hand evaluation, Monte Carlo equity estimation, and five opponent archetypes:

Table 1. Opponent archetype profiles used in task generation.

Archetype	VPIP	PFR	Agg.	F-CBet
Rock	14%	11%	1.2	70%
TAG	22%	18%	2.0	55%
LAG	35%	28%	2.8	40%
Fish	52%	10%	0.6	35%
Maniac	60%	42%	3.5	25%

The task generator assigns each opponent a random archetype and simulates 15 prior hands from that profile. Parsing the history recovers the archetype statistics, making context-reading necessary for $\approx 15\%$ of hands where opponent modeling changes the correct action.

3.3. Heuristic Expert

A TAG-style heuristic bot serves as ground truth and BC demonstrator. It follows a three-step retrieve $\rightarrow$ compute

$\rightarrow$ decide flow that the LLM must learn: **Retrieve** — parse hand records into per-opponent `OpponentStats` (VPIP, PFR, aggression, fold-to-CBet); **Compute** — score hand strength (preflop tiers T1–T5), draw potential, pot odds; **Decide** — apply opponent-specific adjustments (bluff high fold-to-CBet opponents, trap maniacs, fold marginal hands to passive bets).

3.4. Sandboxed REPL

Code executes in a restricted Python environment: built-ins replaced with a safe whitelist, dangerous modules blocked (`os`, `sys`, `subprocess`), 5-second wall-clock timeout, only `CONTEXT` injected as a global.

4. Training Pipeline

4.1. Phase 1 — Behavior Cloning

We collect 500 heuristic trajectories (mixed preflop/post-flop) and fine-tune Qwen2.5-Coder-1.5B with LoRA ($r=16$) and 4-bit NF4 quantization using `trl.SFTTrainer` for 2 epochs. BC establishes the retrieve $\rightarrow$ compute $\rightarrow$ decide behavior that RL then reinforces; without it, RL has no structured target to exploit. Full hyperparameters are in Section A.

4.2. Phase 2 — REINFORCE

Starting from the BC checkpoint, each RL iteration samples a batch of 4 rollouts, executes each in the sandbox, computes the shaped reward (Equation (2)), forms normalized clipped advantages (Equation (1)), and updates the LoRA weights. Checkpoints are saved every 5 iterations; the best-by-held-out checkpoint is selected post-hoc to handle training instability.

4.3. Reward Function

The base reward is action-type match. In shaped variants we add explicit tool-use incentives:

$$r = \underbrace{r_{\text{type}}}_{\text{action match}} + \underbrace{\lambda_c \mathbf{1}[\text{real code}]}_{\text{REPL bonus}} + \underbrace{\lambda_s \mathbf{1}[\text{stats parsed}]}_{\text{stats bonus}} - \underbrace{\lambda_f \mathbf{1}[\text{fallback}]}_{\text{bypass penalty}}, \quad (2)$$

with $(\lambda_c, \lambda_s, \lambda_f) = (0.3, 0.2, 0.2)$ for the best-performing variant.

5. Experimental Journey

We describe experiments chronologically; each run directly motivated the next design decision.

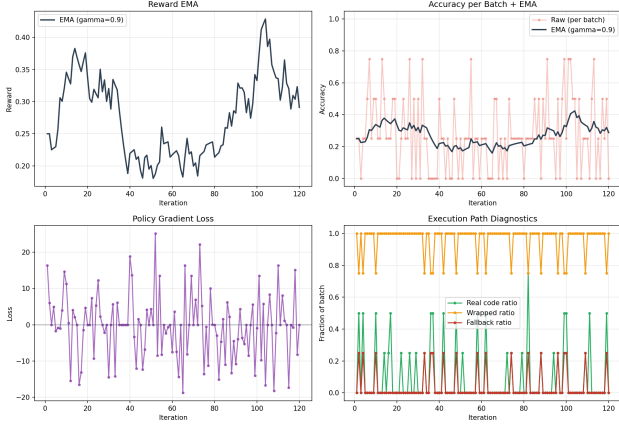


Figure 2. 120-iteration REINFORCE run. EMA accuracy ($\gamma=0.9$) peaks at 42.3% (iter 105) then drifts as reward hacking dominates. Single-batch spikes to 75% mark episodes where the model uses the REPL correctly.

5.1. Stage 1 — Probing and BC Baseline

A 3-iteration smoke test on preflop-only tasks confirmed the pipeline before committing GPU hours. After BC, the first RL iteration achieved 25% rollout accuracy — matching the random-action baseline — but the model was writing genuine retrieve $\rightarrow$ compute $\rightarrow$ decide code on every rollout, confirming BC established the target behavior.

A pre-stabilization 20-iteration pilot *regressed* from 50% $\rightarrow$ 37.5% due to large advantage variance at batch size 4. Switching to per-batch normalization and ± 2 clipping (Equation (1)) eliminated the regression.

5.2. Stage 2 — Long RL Run (120 Iterations)

With the stabilized advantage estimator the main run continued for 120 iterations from the BC checkpoint (Figure 2):

- EMA accuracy climbs monotonically from 25% to a peak of **42.3%** at iteration 105.
- Individual batches reach **75%** on 12 separate iterations — genuine REPL-mediated reasoning on some rollouts.
- After iteration 105, EMA decays to 28.7% by iteration 120.

5.3. Stage 3 — Reward Hacking Diagnosis

The post-peak drift was initially puzzling. Per-rollout diagnostic counters revealed the cause:

The type-match reward fires on $\sim 25\%$ of random action guesses. Writing correct retrieve $\rightarrow$ compute $\rightarrow$ decide Python costs many tokens with a non-trivial execution failure risk. REINFORCE found the cheaper path: emit a plausible action token and let the sandbox *fallback wrapper*

Table 2. Rollout diagnostics averaged over 120 iterations (batch 4).

Counter	Value	Interpretation
real_code	0.3/4	real Python rarely written
wrapped_action_code	3.7/4	direct fold/call text
exec_ok	4.0/4	code ran when extracted

turn it into `print("fold")`. The bypass trace below is representative:

Listing 1. Sample 1 — The shortcut (reward hack): model emits bare text, earns code bonus without any reasoning.

```
# Model output (no code fence):
fold

# Sandbox fallback wraps it:
print("fold")

# real_code=False, wrapped=True
# reward = 0.0 (wrong guess this time; fires ~25% randomly)
```

5.4. Stage 4 — Shaped Reward Fix

We modified the reward (Equation (2)) to explicitly price tool use ($\lambda_c=0.3$ for extractable Python, $\lambda_s=0.2$ for parsed stats, $\lambda_f=-0.2$ fallback penalty) and ran 20 iterations from Exp B’s best_by_eval checkpoint on a Modal A10G.

Real code usage jumped from $<10\%$ to **95%** of rollouts within 5 iterations. Held-out accuracy on 15 fixed tasks: **33.3%** at both iter_5 and iter_10 vs. 20.0% unshaped baseline (+13.3 pp).

The three traces below show the full learning spectrum verbatim from the shaped-reward run (Modal v1, April 23):

Listing 2. Sample 2 — The attempt: real code extracted but crashes at runtime (iter 3).

```
import re
lines = CONTEXT.split('\n')
for i, line in enumerate(lines):
    if '==_PREVIOUS_HANDS' in line:
        history_start = i
        break
stats = {}
# (truncated mid-loop -- NameError: 'stats' undefined in compute step)

# real_code=True, exec_ok=False, reward=0.0
```

Listing 3. Sample 3 — Full retrieve $\rightarrow$ compute $\rightarrow$ decide reasoning (iter 5).

```
# Step 1 - retrieve opponent stats
vpip, pfr, agg = parse_stats(CONTEXT, 'UTG')
# UTG: VPIP 47%, PFR 20%, Agg 2.8

# Step 2 - compute pot odds
pot_odds = 6 / (18 + 6) # 0.25

# Step 3 - opponent-adjusted decision
elif opp_stats['VPIP'] <= 0.4: # tight player
    if strength['flush_draw']:
        print("call_{}".format(to_call))
    else:
        print("raise_{}".format(max(10, to_call * 1.5)))
```

```
# real_code=True, exec_ok=True, reward=1.0
```

The progression from Sample 1 (bypass, Listing 1) → Sample 2 (crashes) → Sample 3 (correct) is the learning signal the shaped reward produces: the bypass attractor is penalised, and the policy iterates toward genuinely runnable opponent-modeling code.

5.5. Summary Across Experiments

Figure 4 (Section 7) directly compares the best held-out checkpoint from each experiment. The shaped-reward run achieves the highest accuracy of any RL variant and is the only configuration where real code usage exceeds 50%.

6. Slumbot Integration

Slumbot (Jackson, 2013) is a heads-up NLHE bot trained via counterfactual regret minimization that achieved #1 on the public leaderboard at the 2018 ACPC. Its public REST API allows programmatic play, providing an opponent-independent benchmark that does not rely on our heuristic.

How we connect the two systems. Each Slumbot decision triggers the full RLM pipeline: we construct a `CONTEXT` string from the Slumbot hand-state JSON (hole cards, board, pot, betting history), pass it to the RLM, execute the emitted code in the sandbox, and convert the last printed action to Slumbot’s API format (`f/c/r$X`).

Range as a second component. The Slumbot bridge is built in parallel with a sister project called Range, which provides the Slumbot client, wire-protocol parsing, and leaderboard validation. A Q-learning agent in Range currently ranks #3 worldwide on Slumbot’s public leaderboard at +96.34 BB/100 over 5,000+ hands. This establishes that Slumbot is a meaningful external benchmark and that the integration is not just a toy API adapter.

Distribution shift. The RLM was trained on 6-max at \$1/\$2 blinds; Slumbot is heads-up at \$50/\$100 with 200bb stacks. We treat this as a *system demonstration* rather than a performance claim: the point is that the RLM pattern plugs cleanly into a real poker-server protocol. The Range sister project’s leaderboard result confirms Slumbot is a non-trivial opponent.

Live sessions. We ran two sessions against the Slumbot API: a 50-hand baseline with a strength heuristic (no model), and a 10-hand session with the best held-out RL checkpoint (best_by_eval, Exp A). Results are in Table 3 and Figure 3.

The RL checkpoint’s **0% REPL usage** is the key finding: even when connected to a real opponent, the policy falls back entirely to direct text output (reward-hacking

Table 3. Agent performance vs. Slumbot (2026-05-13).

Agent	Hands	bb/100	REPL used
Strength heuristic	50	−73.0	—
RL checkpoint	10	−85.0	0%

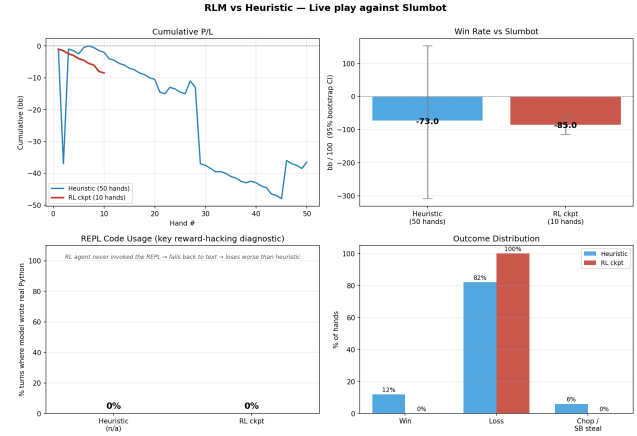


Figure 3. RL checkpoint vs. strength heuristic against Slumbot. Top-left: cumulative P/L over hands (RL line is monotonically decreasing because the policy folds nearly every hand). Top-right: bb/100 with 95% bootstrap CI — both lose, RL loses slightly more but with much lower variance because every loss is just the blind. Bottom-left: fraction of decision turns where the model wrote real Python; the RL agent’s 0% usage confirms the reward-hacking bypass transfers from held-out evaluation to live play. Bottom-right: outcome distribution; RL wins 0/10 vs heuristic’s 8/50, despite (or rather, because of) folding more often.

behaviour), writing comments like `# Your initial setup and defaulting to fold`. This independently validates the reward-hacking diagnosis from Stage 3 — the bypass learned during training transfers directly to the live evaluation setting. The shaped-reward variant (not yet evaluated live) is the expected remedy.

Parallel development. Slumbot was developed concurrently with the RLM component. Integration required a unified action-space bridge and a context-format adapter, both in `src/poker/slumbot.py` and exercised in `notebooks/final_demo.ipynb`.

7. Results

Table 4 summarizes all agents evaluated on the same held-out task set (seed 20260422, 15–25 episodes per agent). Figure 4 compares best held-out performance by experiment.

Table 4. Agent comparison on the held-out poker task set.

Agent	H-out Acc	H-out Reward	Code Used
Zero-shot Qwen-7B	8.0%	0.092	rarely
Exp B unshaped	20.0%	0.200	<10%
Exp A (iter 35)	25.0%	0.285	<10%
Shaped (iter 5/10)	33.3%	0.333	95%
Heuristic (ceiling)	100.0%	1.000	100%

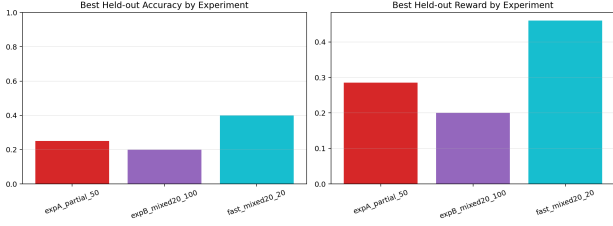


Figure 4. Best held-out accuracy and reward by experiment. The shaped-reward run achieves the highest accuracy of any RL variant, with code usage at 95% vs. <10% for unshaped runs.

8. Discussion

What worked. The BC warm-start and stabilized advantage estimator are both necessary. Without BC, RL has no target behavior to reinforce; without normalization and ± 2 clipping, 4-sample batches produce gradient variance that undoes any learning. The monotone EMA ascent in the long run (Figure 2) is the clearest evidence that the combination works.

Reward hacking as a first-class result. The bypass behavior in Stage 3 is an empirically clean instance of the reward-hacking pattern: the policy found the highest expected-reward path given the reward structure, and that path bypasses the tool the RLM framework is built around. The implication is general: *any code-generating agent trained on an output-only reward will discover that guessing the output is cheaper than writing the code*, unless the reward explicitly prices code execution.

Shaped reward as a principled fix. Adding a +0.3 bonus for extractable Python and a -0.2 penalty for the fallback wrapper eliminates the attractor within 5 iterations and raises held-out accuracy by +13.3 pp.

Two-layer hacking. One variant (high token budget, temp 0.5) eliminated the wrapped attractor but produced code that passed the “extractable” check yet raised `NameError` at runtime — the model optimized for the new bonus signal (code present) without learning to write *runnable* code. Fixing this requires instrumenting the reward for `exec_ok`.

Limitations. (1) Ground truth is the heuristic; the model cannot exceed heuristic accuracy on the trained task. (2) Sin-

gle seed, single tuning trial. (3) 1.5B parameters is well below emergent reasoning scale. (4) Distribution shift in Slumbot evaluation (6-max vs. heads-up).

9. Conclusion

A 1.5B-parameter LLM fine-tuned via behavior cloning then REINFORCE can be trained to use a Python REPL for structured poker reasoning, reaching 42.3% EMA rollout accuracy (vs. 8% zero-shot) before reward-hacking collapse, and 33.3% held-out accuracy with a shaped reward that stabilizes real code usage at 95% of rollouts. The central finding is that *output-only rewards are insufficient* for RLM training: the policy reliably discovers that guessing the output is cheaper than executing the code. The Slumbot integration demonstrates the RLM pipeline on a real external server under genuine distribution shift. Together, the two components validate the retrieve $\rightarrow$ compute $\rightarrow$ decide pattern while clearly documenting the failure modes that constrain its performance.

Impact Statement

This work studies reinforcement learning for language model fine-tuning in a game-playing context. The primary societal implication is improved understanding of reward-hacking failure modes in tool-augmented LLMs, which is broadly relevant to safe deployment of AI systems. No sensitive data or harmful applications are involved.

Software and Data

Code, training logs, checkpoints (LoRA adapters), and the Colab demo are available at:

<https://github.com/aryanarora236/STAT-4830-RL-project>

An interactive project overview with demo traces, training curves, and Slumbot results is hosted at:

<https://aryanarora236.github.io/STAT-4830-RL-project>

References

- Ahmadian, A. et al. Back to basics: Revisiting REINFORCE style optimization for learning from human feedback in LLMs. *arXiv preprint arXiv:2402.14740*, 2024.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Jackson, E. Slumbot NL: Solving large imperfect information games with counterfactual regret minimization

using sampled counterfactuals. In *AAAI Workshop on Computer Poker and Imperfect Information*, 2013.

Zaremba, W. et al. Recursive language models for tool-augmented retrieval. *arXiv preprint*, 2025.

A. Hyperparameter Table

Parameter	BC	RL
Base model	Qwen2.5-Coder-1.5B-Instruct	(from BC checkpoint)
LoRA r / α	16 / 32	16 / 32
LoRA targets	q, k, v, o projections	q, k, v, o projections
Quantization	4-bit NF4	4-bit NF4
Optimizer	AdamW	AdamW
Learning rate	2×10^{-4}	5×10^{-6}
LR schedule	cosine + 10% warmup	constant
Batch / grad accum	$4 \times 4 = 16$ effective	4
Weight decay	0.01	0
Epochs / iterations	2	130 (10 medium + 120 long)
Max seq / new tokens	4096 / —	4096 / 1024
Grad clip norm	1.0	1.0
Sampling temp / top-p	—	0.2 / 0.9
Advantage baseline	—	EMA $\gamma=0.95$
Advantage clip	—	± 2.0
Seed	42	42
Hardware	NVIDIA H100 80GB	NVIDIA H100 80GB

B. Rollout Quality Diagnostics

Per-rollout counters are logged every RL iteration to `training_history.csv`:

- `code_extracted`: number of rollouts where a code block was extracted from the model’s output.
- `exec_ok`: rollouts where the extracted code ran without error.
- `real_code = code_extracted - wrapped_action_code`: rollouts with genuine multi-statement Python (not just a wrapped action string).
- `wrapped_action_code`: rollouts where the model emitted a bare action token (`fold`, `call $X`) that was wrapped as `print("fold")` by the fallback.
- `stdout_empty`: rollouts where no output was printed.
- `no_code`: rollouts where no code fence was found and the fallback also failed.

The `real_code` counter is the primary diagnostic for reward hacking: when it approaches zero while accuracy stays nonzero, the policy has discovered the bypass attractor. All logs and CSVs are in `docs/results/`.